

Sur le problème
d'isomorphisme de groupes

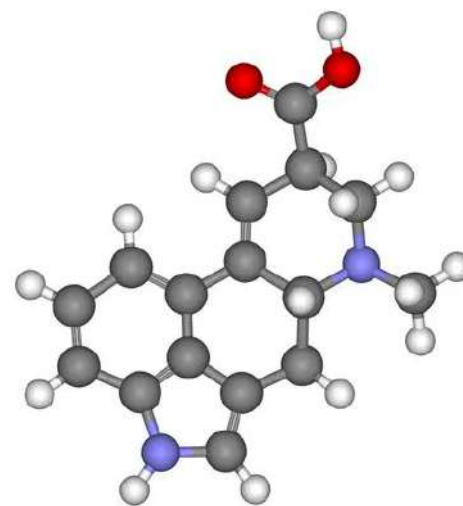
**JEON Saehyun Davin n°
29566**

Partenaire: BOURILLON Jules
n°44227

Introduction



Cryptographie



Chimie moléculaire

Cadre de résolution

Énoncé du problème

IsoGroupe

Entrée : $n \in \mathbb{N}^*$, deux groupes G_1 et G_2 (qu'on identifie à $\llbracket 0, n-1 \rrbracket$) tels que $|G_1| = |G_2| = n$, donnés par leurs tables de Cayley.

Sortie :

$$\begin{cases} \text{Non} & \text{si } G_1 \not\cong G_2, \\ (\text{Oui}, \phi) & \text{où } \phi : G_1 \rightarrow G_2 \text{ est un isomorphisme explicite} \end{cases}$$

Motivation:

- **Accès en $O(1)$ à l'ordre du groupe**
- **Calculs en $O(1)$**
- **Facile de coder soit même des exemples de groupes**

Plan

**I) Implémentation pour le cas général:
l'algorithme de Tarjan**

**II) Le cas particulier des groupes abéliens:
l'algorithme de Nader H. Bshouty**

III) Analyse des résultats

I - Implémentation pour le cas général: l'algorithme de Tarjan

I - Implémentation pour le cas général

Un algorithme naïf ?

NAÏF

Entrée : Deux groupes G et G' .

Sortie : VRAI si $G \cong G'$, FAUX sinon.

Pour chaque bijection $f : G \rightarrow G' :$

Si $\forall (x, y) \in G \times G, f(x \cdot y) = f(x) \cdot f(y) :$

Renvoyer VRAI

Renvoyer FAUX

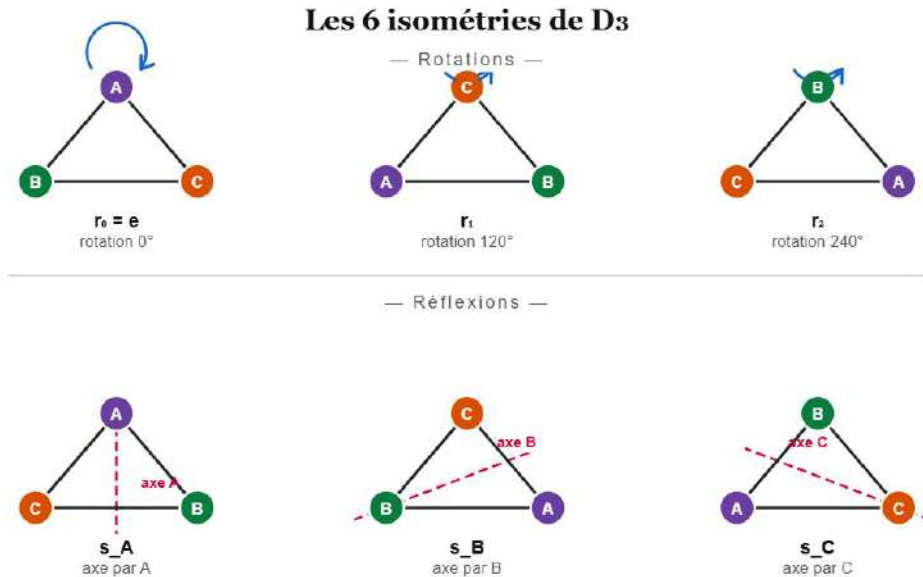
Complexité: $\Omega(n^2n!)$ (même en haut niveau)...

Solution ? Utiliser des générateurs.

I - Implémentation pour le cas général

Etude d'un cas concret

Le groupe D_3 des isométries du triangle

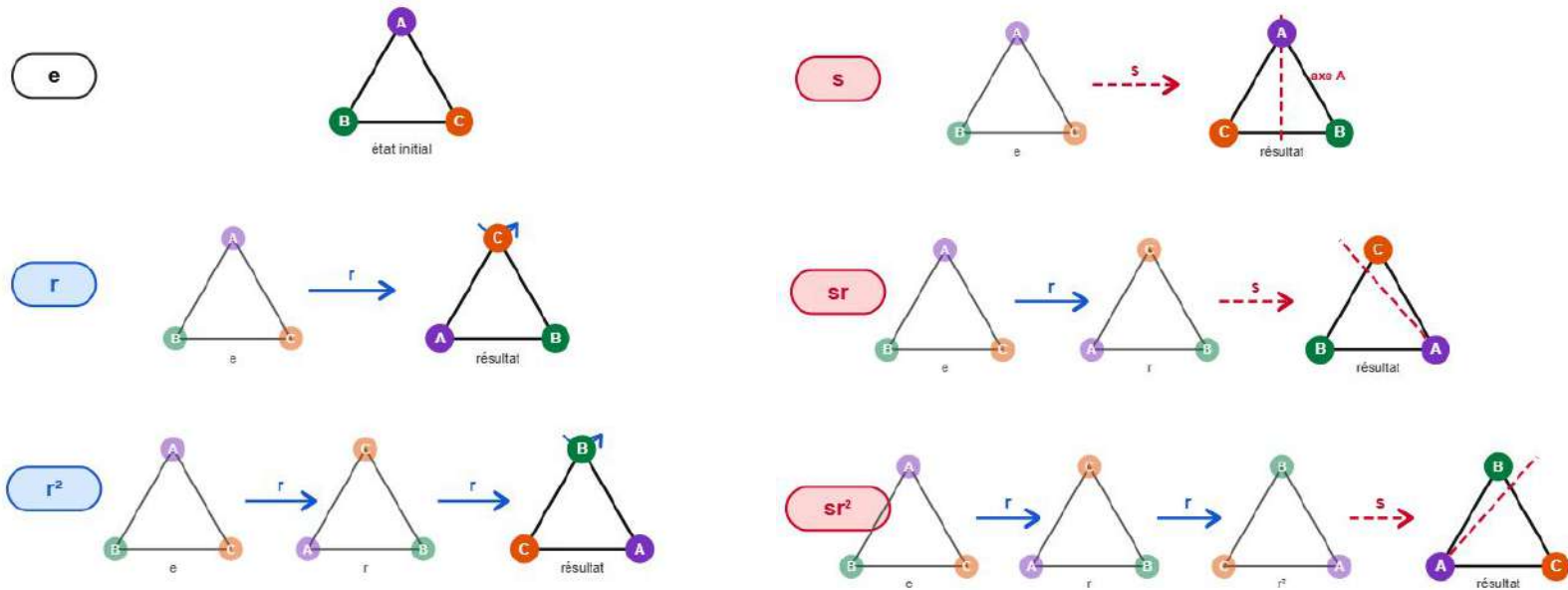


Le groupe S_3 , des permutations de $\{1, 2, 3\}$

Permutation	Tableau	Effet sur $[1,2,3]$
$()$	$[1,2,3]$	$[1,2,3]$
$(1\ 2\ 3)$	$[1,2,3]$	$[2,3,1]$
$(1\ 3\ 2)$	$[1,2,3]$	$[3,1,2]$
$(1\ 2)$	$[1,2,3]$	$[2,1,3]$
$(2\ 3)$	$[1,2,3]$	$[1,3,2]$
$(1\ 3)$	$[1,2,3]$	$[3,2,1]$

I - Implémentation pour le cas général

Etude d'un cas concret



D3 est généré par deux éléments:
 $r = r^{120}$ et $s = s_A$

I - Implémentation pour le cas général

Etude d'un cas concret

Décomposition	Permutation equivalente	Effet sur [1,2,3]
Id	()	[1,2,3]
r'	(1 2 3)	[2,3,1]
$r' * r'$	(1 3 2)	[3,1,2]
s'	(1 2)	[2,1,3]
$s' * r'$	(1 3)	[3,2,1]
$s' * r' * r'$	(2 3)	[1,3,2]

S_3 est généré par deux éléments: $r' = (1\ 2\ 3)$ et $s' = (1\ 2)$

I - Implémentation pour le cas général

Etude d'un cas concret

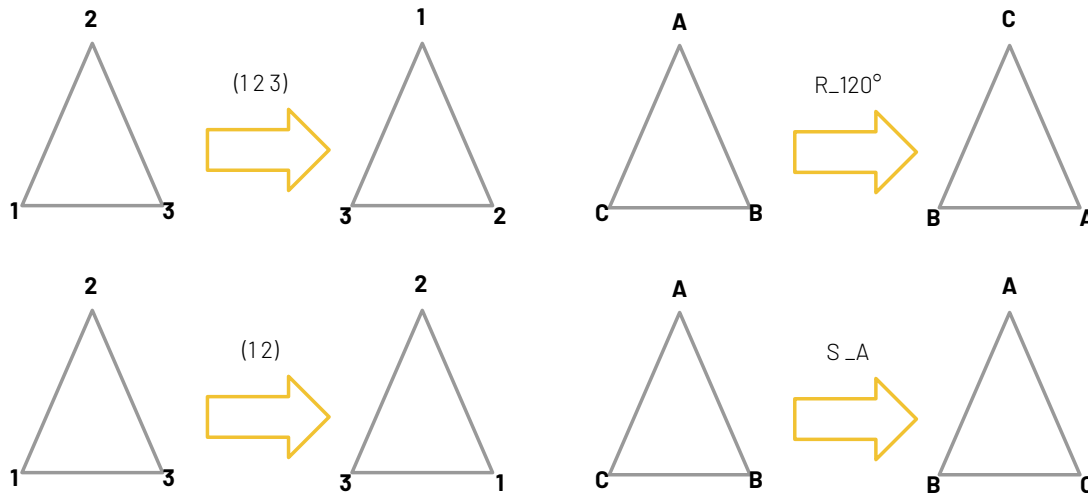
Décomposition	Permutation equivalente	Effet sur [1,2,3]
Id	()	[1,2,3]
r'	(1 2 3)	[2,3,1]
$r' * r'$	(1 3 2)	[3,1,2]
s'	(1 2)	[2,1,3]
$s' * r'$	(1 3)	[3,2,1]
$s' * r' * r'$	(2 3)	[1,3,2]

S_3 est généré par deux éléments: $r' = (1\ 2\ 3)$ et $s' = (1\ 2)$

Correspondance exacte des générateurs et des décompositions.

I - Implémentation pour le cas général

Etude d'un cas concret



$f: S_3 \rightarrow D_3$

$r' = (1\ 2\ 3) \rightarrow r$

$s' = (1\ 2) \rightarrow s$

**s'étend en un
isomorphisme
de S_3 dans D_3**

Pour calculer l'image de $(1\ 3\ 2)$:

$$f((1\ 3\ 2)) = f(r' \circ r') = f(r') \circ f(r') = r \circ r = r_{240^\circ}$$

I - Implémentation pour le cas général

L'approche de Tarjan

SONTISOMORPHES

Entrée : Deux groupes G et G' de même ordre, $F \subset G$ une famille de générateurs.

Sortie : (VRAI, ϕ) si G et G' sont isomorphes, FAUX sinon.

Pour chaque application injective $f : F \rightarrow G'$:

 Étendre f en une application $\phi : G \rightarrow G'$

Si ESTISOMORPHISME(ϕ, G, G') :

Renvoyer (VRAI, ϕ)

Renvoyer FAUX

I - Implémentation pour le cas général

L'approche de Tarjan

SONTISOMORPHES

Entrée : Deux groupes G et G' de même ordre, $F \subset G$ une famille de générateurs.

Sortie : (VRAI, ϕ) si G et G' sont isomorphes, FAUX sinon.

Pour chaque application injective $f : F \rightarrow G'$:

 Étendre f en une application $\phi : G \rightarrow G'$

Si ESTISOMORPHISME(ϕ, G, G') :

Renvoyer (VRAI, ϕ)

Renvoyer FAUX

Avec: $m = |F|$, $n = |G| = |G'|$

Nombre d'applications injectives de F dans G : $\frac{n!}{(n-m)!}$

$\left. \begin{array}{l} \text{Étendre } f \text{ en } \phi : G \rightarrow G' : O(nm) \\ \text{Vérifier l'isomorphisme : } O(n^2) \end{array} \right\} O(n^2)$

I - Implémentation pour le cas général

L'approche de Tarjan

SONTISOMORPHES

Entrée : Deux groupes G et G' de même ordre, $F \subset G$ une famille de générateurs.

Sortie : (VRAI, ϕ) si G et G' sont isomorphes, FAUX sinon.

Pour chaque application injective $f : F \rightarrow G'$:

Étendre f en une application $\phi : G \rightarrow G'$

Si ESTISOMORPHISME(ϕ, G, G') :

Renvoyer (VRAI, ϕ)

Renvoyer FAUX

Coûte
 $O(nm) = O(n^2)$

Gain d'un facteur
 $(n-m)!$

Avec: $m = |F|$, $n = |G| = |G'|$

Nombre d'applications injectives de F dans G : $\frac{n!}{(n-m)!}$

$\left. \begin{array}{l} \text{Étendre } f \text{ en } \phi : G \rightarrow G' : O(nm) \\ \text{Vérifier l'isomorphisme} : O(n^2) \end{array} \right\} O(n^2)$

But: trouver une famille de générateurs de petite taille.

I - Implémentation pour le cas général

i) Trouver une famille de générateurs

GÉNÉRATEURS

Entrée : Un groupe fini G d'ordre n .

Sortie : Une famille F de générateurs avec $|F| = O(\log n)$ et R un tableau de listes chaînées des décompositions.

$F \leftarrow \emptyset$

$H \leftarrow \emptyset$

Tant que $H \neq G$:

$x \leftarrow$ un élément de $H \setminus G$

$F \leftarrow F \cup \{x\}$

Pour $y \in H$:

$H \leftarrow H \cup \{x \cdot y\}$

$R[xy] \leftarrow x \cdot R[y]$

Renvoyer F, R

H double à chaque étape $\Rightarrow |F| = O(\log n)$

Complexité: $O(n \log(n))$

I - Implémentation pour le cas général

ii) Complexité finale

SONTISOMORPHES

Entrée : Deux groupes G et G' de même ordre, $F \subset G$ une famille de générateurs.

Sortie : (VRAI, ϕ) si G et G' sont isomorphes, FAUX sinon.

Pour chaque application injective $f : F \rightarrow G' :$

 Étendre f en une application $\phi : G \rightarrow G'$

Si ESTISOMORPHISME(ϕ, G, G') :

Renvoyer (VRAI, ϕ)

Renvoyer FAUX

Complexité pour étendre f en $\phi : O(n \times |F|) = O(n \log(n))$

Complexité de EstIsomorphisme : $O(n^2)$

Complexité totale d'un passage de boucle: $O(n^2)$

Nombre d'applications $f : G \rightarrow G'$ injectives : $O\left(\frac{n!}{(n - \lfloor \log(n) \rfloor)!}\right)$

I - Implémentation pour le cas général

iii) Complexité finale

Après calculs, on trouve, avec la formule de Stirling: $n! \sim \sqrt{2\pi n} \left(\frac{n}{e}\right)^n$

$$O\left(\frac{n!}{(n - \lfloor \log(n) \rfloor)!}\right) = O(n^{\log(n)+2.45})$$

Complexité finale: $n^{\log(n)+O(1)}$

I - Implémentation pour le cas général

Implémentation: structure de groupe

```
13  typedef struct group {
14      char *name; // Nom
15      int n; // Cardinal du groupe
16      int **table; /* Table de Cayley:
17                  |   |   |   |   |   |   |   le groupe est assimilé
18                  |   |   |   |   |   |   |   à [0, 1, ..., n-1]
19                  |   |   |   |   |   |   |   */
20  } group_t;
```

Espace mémoire: $O(n^2)$

Calcul: $O(1)$

I - Implémentation pour le cas général

Implémentation: structures pour les calculs

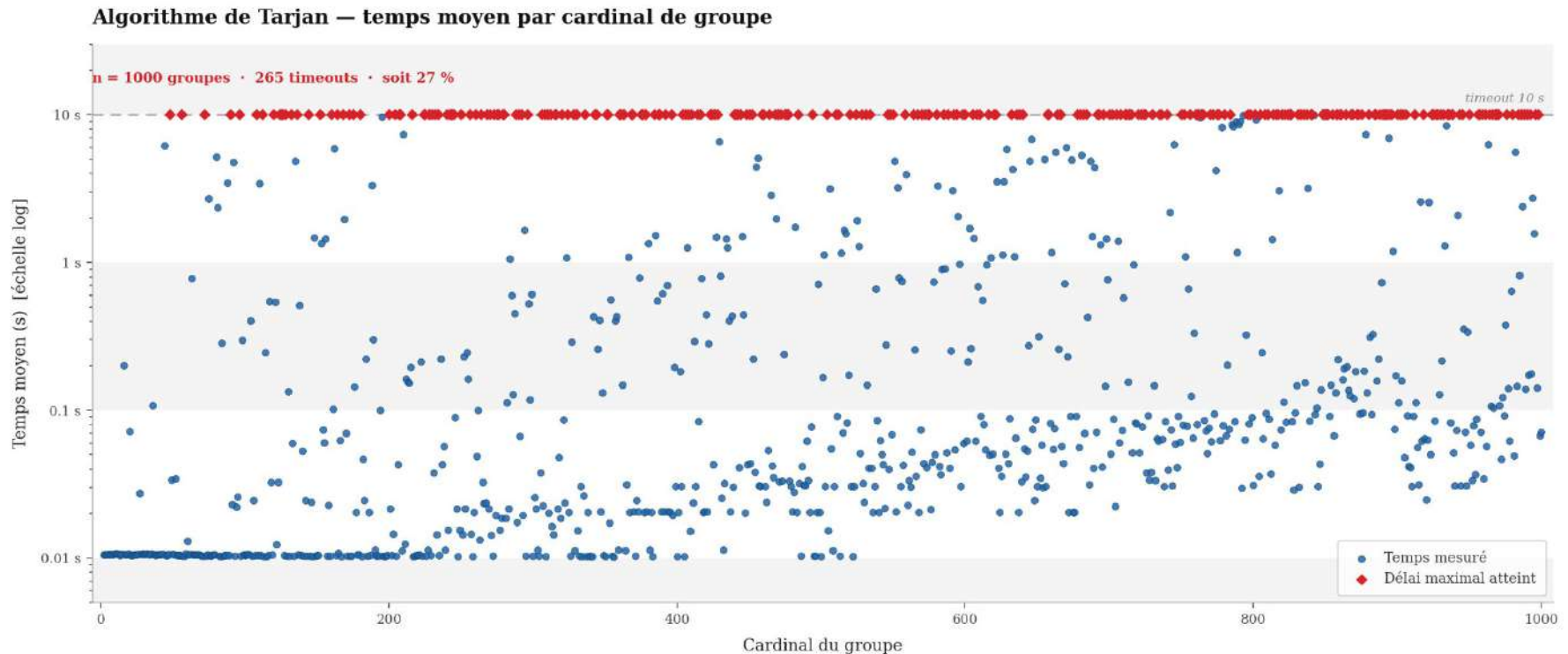
```
9  typedef struct node {
10     int val;
11     struct node* next;
12 } node_t;
13
14 typedef struct list {
15     node_t* head;
16     node_t* tail;
17     int length;
18 } list_t;
```

```
6  typedef struct gentabl {
7     int size;
8     int num_elts;
9     list_t** decomp; /* Table de décomposition des
10                      | éléments du groupe
11                      | selon les générateurs
12                      | */
13 } gentabl_t;
```

```
22 typedef struct group_data {
23     gentabl_t* rel; // Table des décompositions
24     list_t* gen; // Liste des générateurs
25 } gdata_t;
```

I - Implémentation pour le cas général

Résultats expérimentaux

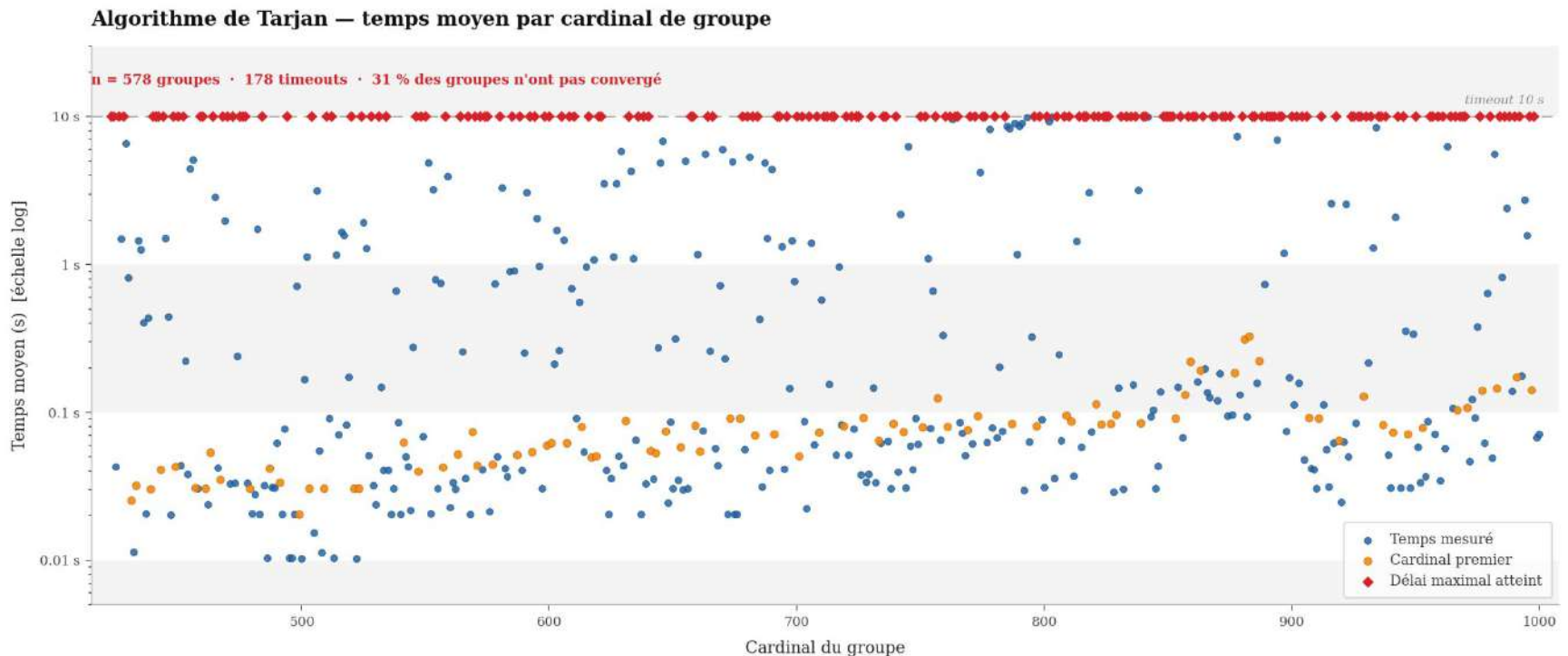


Bleu: temps moyen mesuré (sur dix itérations)

Rouge: délai maximal de 10 secondes atteint - 265 dépassements sur 1000 groupes soit 27%

I - Implémentation pour le cas général

Résultats expérimentaux



Bleu: temps moyen mesuré (sur dix itérations)

Rouge: délai maximal de 10 secondes atteint - 265 dépassements sur 1000 groupes soit 27%

Orange: groupe de cardinal premier - pas très efficace même quand le groupe est cyclique

II - Le cas particulier des groupes abéliens: l'approche de Nader H. Bshouty

II - Le cas particulier des groupes abéliens

Résultat théorique important.

Théorème de structures des groupes abéliens finis:

Soit G un groupe abélien fini. Alors G est isomorphe à un unique produit de groupes cycliques:

$$G \cong \mathbb{Z}/d_1\mathbb{Z} \times \dots \times \mathbb{Z}/d_r\mathbb{Z}$$

où les d_i vérifient: $d_1 \mid \dots \mid d_r$ et les $d_i > 1$

Objectif: trouver les (d_i)

II - Le cas particulier des groupes abéliens

i) Trouver des générateurs et les décompositions

CALCUL GÉNÉRATEURS

Entrée : Un groupe fini G avec $n = |G|$.

Sorties : Les générateurs $\{a_1, \dots, a_t\}$ avec $t \leq \log_2(n)$
et une table de listes chaînées $(\lambda(x))_{x \in G}$ des décompositions selon les générateurs

$A_0 \leftarrow \emptyset$

$G_0 \leftarrow \{e\}$

$i \leftarrow 0$

$\forall a \in G, \lambda(a) \leftarrow \text{NULL}$

Tant que $G \neq G_i$:

$i \leftarrow i + 1$

$G_i \leftarrow G_{i-1}$

Choisir $a_i \in G \setminus G_{i-1}$

$A_i \leftarrow A_{i-1} \cup \{a_i\}$

Trouver le plus petit k_i tel que $a_i^{k_i} \in G_{i-1}$

Pour $(x, j) \in G_{i-1} \times \{1, \dots, k_i - 1\}$:

$\lambda(a_i^j x) \leftarrow \text{CopierListe}(\lambda(x))$

Ajouter en tête de $\lambda(a_i^j x)$ le nœud (a_i, j, NULL)

$G_i \leftarrow G_i \cup \{a_i^j x\}$

Renvoyer $t \leftarrow i$, $A \leftarrow A_i$, $(k_1, k_2, \dots, k_t)$ et la table de listes chaînées λ

Complexité: $O(n \log(n))$

II - Le cas particulier des groupes abéliens

i) Trouver des générateurs et les décompositions

```
10  typedef struct group {
11      char* name;
12      int n;
13      int** table;
14  } group_t;
15  typedef group_t* group;

9   typedef struct node {
10     int val;
11     struct node* next;
12  } node_t;
13
14  typedef struct list {
15     node_t* head;
16     node_t* tail;
17     int length;
18  } list_t;
19  typedef list_t* list;

10  typedef struct data_s {
11     int g;
12     int p;
13  } data_t;
14  typedef data_t* data;
15
16  typedef struct node_data_s {
17     int g;
18     int p;
19     struct node_data_s* next;
20  } node_data_t;
21
22  typedef node_data_t* node_data;
23
24  typedef struct data_list_s {
25     node_data head;
26     node_data tail;
27     int length;
28  } data_list_t;
29  typedef data_list_t* data_list;

61  typedef struct group_data_s {
62     int n;
63     bool initialised;
64
65     bool* contains;
66     list grp_elts;
67
68     int* order;
69     // (at,kt)->...->(a1,k1)
70     data_list generators;
71
72     data_list* decomp;
73  } group_data_t;
74  typedef group_data_t* group_data;
```

II - Le cas particulier des groupes abéliens

i) Trouver des générateurs et les décompositions

Groupe $\mathbb{Z}/4\mathbb{Z} \times \mathbb{Z}/2\mathbb{Z}$ d'ordre 8

0 1 2 3 4 5 6 7

1 0 3 2 5 4 7 6

2 3 4 5 6 7 0 1

3 2 5 4 7 6 1 0

4 5 6 7 0 1 2 3

5 4 7 6 1 0 3 2

6 7 0 1 2 3 4 5

7 6 1 0 3 2 5 4

Elément neutre: 0

Décompositions:

1: (1,1) -> END

Générateur(s): (1,2) -> END

Nombre d'éléments: 2

1 -> 0 -> NULL

Décompositions:

1: (1,1) -> END

2: (3,1) -> (1,1) -> END

3: (3,1) -> END

4: (3,2) -> END

5: (3,2) -> (1,1) -> END

6: (3,3) -> (1,1) -> END

7: (3,3) -> END

Générateur(s): (3,4) -> (1,2) -> END

Nombre d'éléments: 8

3 -> 2 -> 4 -> 5 -> 7 -> 6 -> 1 -> 0 -> NULL

II - Le cas particulier des groupes abéliens

ii) Calculs des relations triangulaires

En notant $H = \{a_1, \dots, a_t\}$ et en assimilant la liste chaînée $\lambda(x)$ à un tableau $(\lambda_1(x), \dots, \lambda_t(x))$, on a:

$$\text{pour } i \text{ allant de } 2 \text{ à } t: a_i^{k_i} = a_{i-1}^{l_{i,i-1}} \dots a_1^{l_{i,1}}, a_1^{k_1} = e$$

$$\forall x \in G : x = a_1^{\lambda_1(x)} \dots a_t^{\lambda_t(x)}$$

On dispose d'un isomorphisme naturel entre G et $\Gamma = \left(\bigoplus_{i=1}^t \mathbb{Z}x_i \middle/ R \right)$

$$\text{avec } \Phi : a_1^{\lambda_1} \dots a_t^{\lambda_t} \rightarrow \lambda_1 x_1 + \dots + \lambda_t x_t$$

Où: $R = (k_i x_i = k_{i-1} x_{i-1} + \dots + k_1 x_1 \text{ pour } i \text{ allant de } 2 \text{ à } t, k_1 x_1 = 0)$

II - Le cas particulier des groupes abéliens

iii) Réductions des relations

$$R = \begin{bmatrix} -k_t & \ell_{t,t-1} & \ell_{t,t-2} & \cdots & \ell_{t,2} & \ell_{t,1} \\ 0 & -k_{t-1} & \ell_{t-1,t-2} & \cdots & \ell_{t-1,2} & \ell_{t-1,1} \\ 0 & 0 & -k_{t-2} & \cdots & \ell_{t-2,2} & \ell_{t-2,1} \\ \vdots & \vdots & \ddots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & \cdots & 0 & -k_1 \end{bmatrix}.$$

Forme normale de Smith: $URV = \text{diag}(d_1, \dots, d_r, 0, \dots, 0) = D$

En notant $h = \begin{pmatrix} x_t \\ \vdots \\ x_t \end{pmatrix}$ on a:

$$Rh = 0 \iff U^{-1}DV^{-1}h = 0 \iff D(V^{-1}h) = 0$$

II - Le cas particulier des groupes abéliens

ii) Réductions des relations

En notant $y = \begin{pmatrix} y_1 \\ \vdots \\ y_t \end{pmatrix}$ on a:

$$Dy = 0 \iff d_i y_i = 0 \text{ pour } i \text{ allant de } 1 \text{ à } r$$

Les (y_i) forment une base de Γ et on en déduit une base de G .

$$G \cong \mathbb{Z}/d_1\mathbb{Z} \times \dots \times \mathbb{Z}/d_r\mathbb{Z}$$

II - Le cas particulier des groupes abéliens

Complexité totale

Calculs de la FNS - Kanna-Achim Bachem (1979):

pour $A \in M_n(\mathbb{Z})$, et $\sup_{i,j} |a_{i,j}| \leq K$

$$O(n^4 \log(K))$$

Inversion d'une matrice de taille $n \times n$ à coefficients entiers: $O(n^3)$

II - Le cas particulier des groupes abéliens

Complexité totale

Calculs de la FNS - Kanna-Achim Bachem (1979):

pour $A \in M_n(\mathbb{Z})$, et $\sup_{i,j} |a_{i,j}| \leq K$

$$O(n^4 \log(K))$$

Inversion d'une matrice de taille $n \times n$ à coefficients entiers: $O(n^3)$

**Or ici, la matrice est de taille en $m \times m$ avec
 $m = O(\log n)$: tous les calculs sont donc en poly(log n).**

II - Le cas particulier des groupes abéliens

Complexité totale

Pour décider si deux groupes sont isomorphes:

$$\begin{array}{c} \boxed{\begin{array}{l} \text{1 - Calculs des} \\ \text{générateurs/reliations} \\ O(n \log(n)) \end{array}} + \boxed{\begin{array}{l} \text{2 - Réductions des} \\ \text{relations: SNF*} \\ O(\log^6(n)) \end{array}} + \boxed{\begin{array}{l} \text{3 - Comparer les matrices} \\ \text{diagonales} \\ O(\log(n)) \end{array}} \\ = O(n \log(n)) \end{array}$$

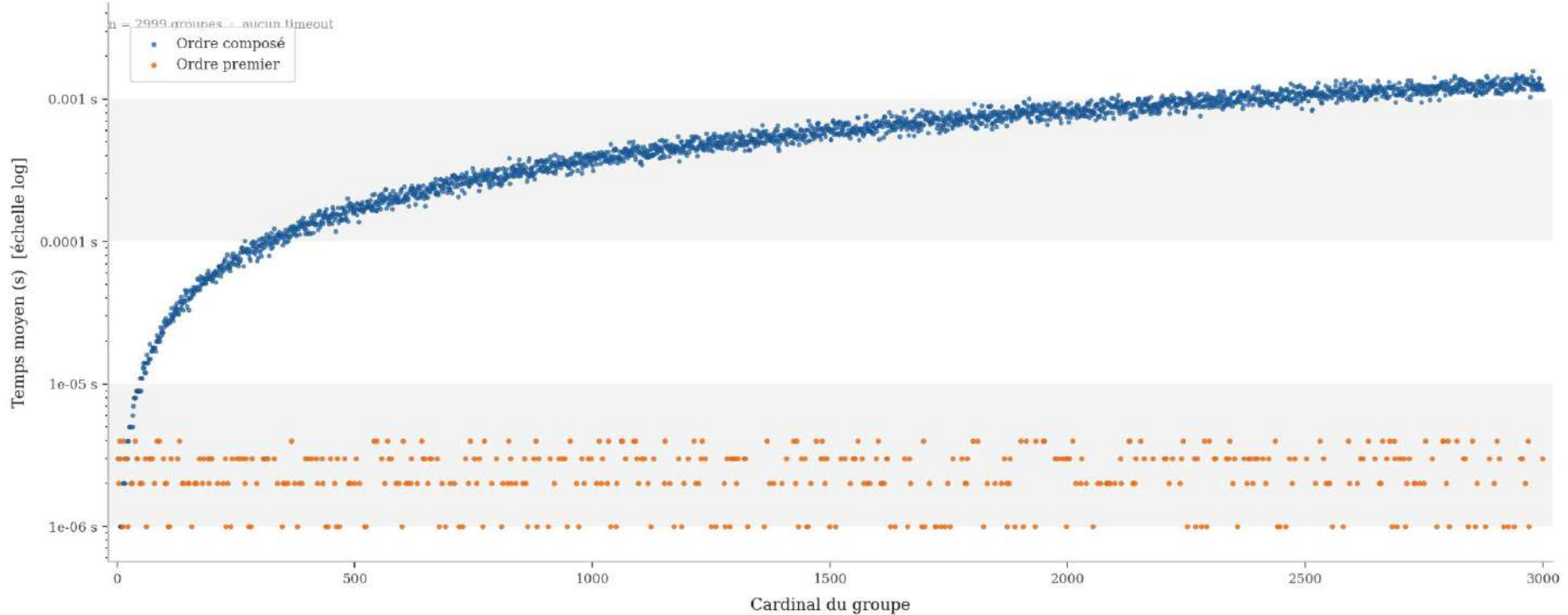
Pour calculer explicitement un isomorphisme:

$$\begin{array}{c} + \boxed{\begin{array}{l} \text{4 - Inverser la matrice V} \\ O(\log^3(n)) \end{array}} + \boxed{\begin{array}{l} \text{5 - Calcul de la base} \\ O(\log^2(n)) \end{array}} + \boxed{\begin{array}{l} \text{6 - Déterminer} \\ \text{l'isomorphisme} \\ O(n \log^2(n)) \end{array}} \\ = O(n \log^2(n)) \end{array}$$

II - Le cas particulier des groupes abéliens

Résultats expérimentaux

Algorithme de Nader H. Bshouty — temps moyen par cardinal de groupe

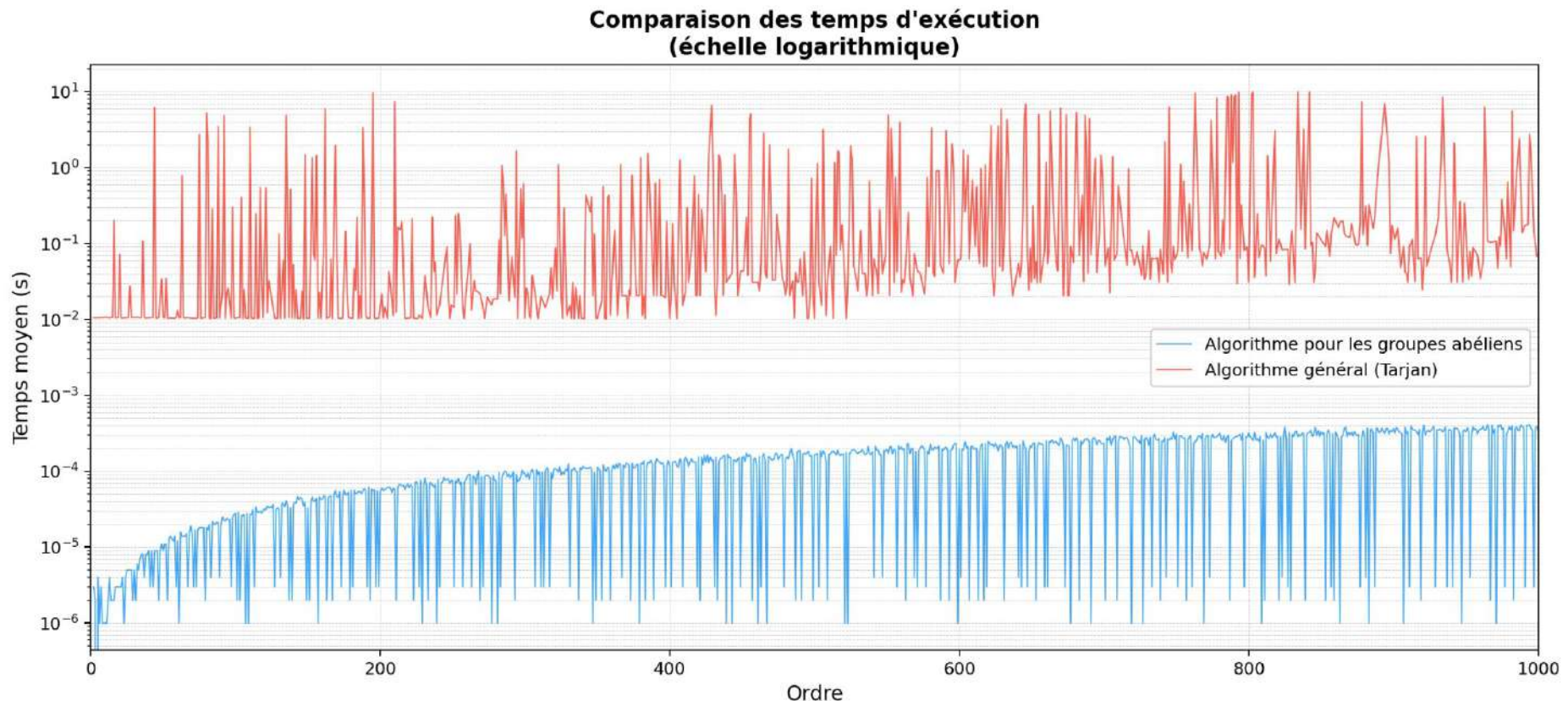


Orange: groupe de cardinal premier, quasi instantané

III - Conclusion et analyse des résultats

III - Analyse des résultats

Temps d'exécution



Algorithme général: beaucoup d'oscillations

Algorithme pour les groupes abéliens: beaucoup moins d'oscillations

Annexe: Amélioration de l'algorithme de Tarjan

Annexe: amélioration de l'algorithme de Tarjan

Complexité totale

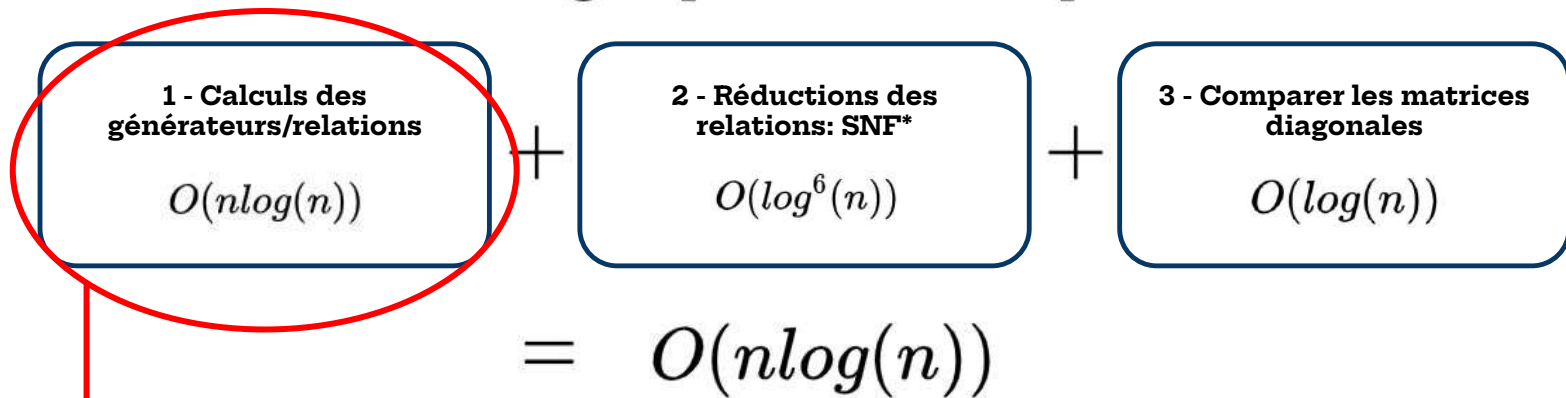
Pour décider si deux groupes sont isomorphes:

1 - Calculs des générateurs/rerelations $O(n \log(n))$	+	2 - Réductions des relations: SNF* $O(\log^6(n))$	+	3 - Comparer les matrices diagonales $O(\log(n))$
$= O(n \log(n))$				

Annexe: amélioration de l'algorithme de Tarjan

Complexité totale

Pour décider si deux groupes sont isomorphes:



Choisir $a_i \in G \setminus G_{i-1}$
 $A_i \leftarrow A_{i-1} \cup \{a_i\}$
Trouver le plus petit k_i tel que $a_i^{k_i} \in G_{i-1}$
Pour $(x, j) \in G_{i-1} \times \{1, \dots, k_i - 1\}$:
 $\lambda(a_i^j x) \leftarrow \text{CopierListe}(\lambda(x))$
Ajouter en tête de $\lambda(a_i^j x)$ le nœud (a_i, j, NULL)

Coût:
 $O(\log(n))$

Annexe: amélioration de l'algorithme de Tarjan

Solution

CALCUL GÉNÉRATEURS

Entrée : Un groupe fini G avec $n = |G|$.

Sorties : Les générateurs $\{a_1, \dots, a_t\}$ avec $t \leq \log_2(n)$ et une table de listes chaînées $(\lambda(x))_{x \in G}$ des décompositions selon les générateurs

```
A0 ← ∅
G0 ← {e}
i ← 0
∀a ∈ G, λ(a) ← NULL
Tant que G ≠ Gi :
  i ← i + 1
  Gi ← Gi-1
  Choisir ai ∈ G \ Gi-1
  Ai ← Ai-1 ∪ {ai}
  Trouver le plus petit ki tel que aiki ∈ Gi-1
  Pour (x, j) ∈ Gi-1 × {1, ... ki - 1} :
    λ(aijx) ← CopierListe(λ(x))
    λ(aijx) ← nouveau nœud(ai, j, next → λ(x))
  Gi ← Gi ∪ {aijx}
Renvoyer t ← i, A ← Ai, (k1, k2, ..., kt) et la table de listes
chaînées λ
```

Coût:
 ~~$\Theta(\log(n))$~~
 $O(1)$

Complexité: $O(n)$

Annexe: amélioration de l'algorithme de Tarjan

Complexité totale

Pour décider si deux groupes sont isomorphes:

$$\begin{array}{c} \boxed{\begin{array}{l} \text{1 - Calculs des} \\ \text{générateurs/relations} \\ \cancel{O(n \log(n))} \\ O(n) \end{array}} + \boxed{\begin{array}{l} \text{2 - Réductions des} \\ \text{relations: SNF*} \\ O(\log^6(n)) \end{array}} + \boxed{\begin{array}{l} \text{3 - Comparer les matrices} \\ \text{diagonales} \\ O(\log(n)) \end{array}} \\ = \cancel{O(n \log(n))} \\ \boxed{O(n)} \end{array}$$

On peut décider si deux groupes abéliens, finis, sont isomorphes en temps linéaire.

Annexe: Code source